

Intro to Relational Model + It's Mapping from ER Diagram

Assumptions:

- Binary relations are sufficient
- The slides here use a notation more faithful to Chen's notation
 - You must use the CSCI3170 notation for assessments
 - Difference specifically on denoting cardinality only

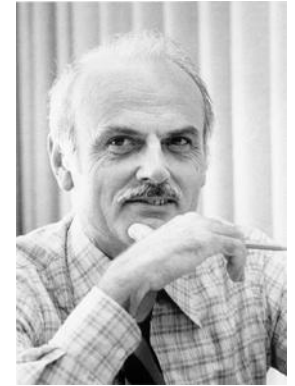
History:

- The relational model was proposed by **Codd** in 1970 (At that time most databases were based on the hierarchical model or the network model)
- Prototype relational DBMSs were developed in IBM and UC-Berkeley by the mid-1970s
- The relational model attracted a lot of attention at that time because it is simple and elegant
- Today, the relational model is by far the dominant data model and the foundation for the leading DBMS; main component for representing data in the relational model is a set of relations

(1) Rel. Model Basics

Deriving a Relational Diagram from an ER Diagram

Introduction

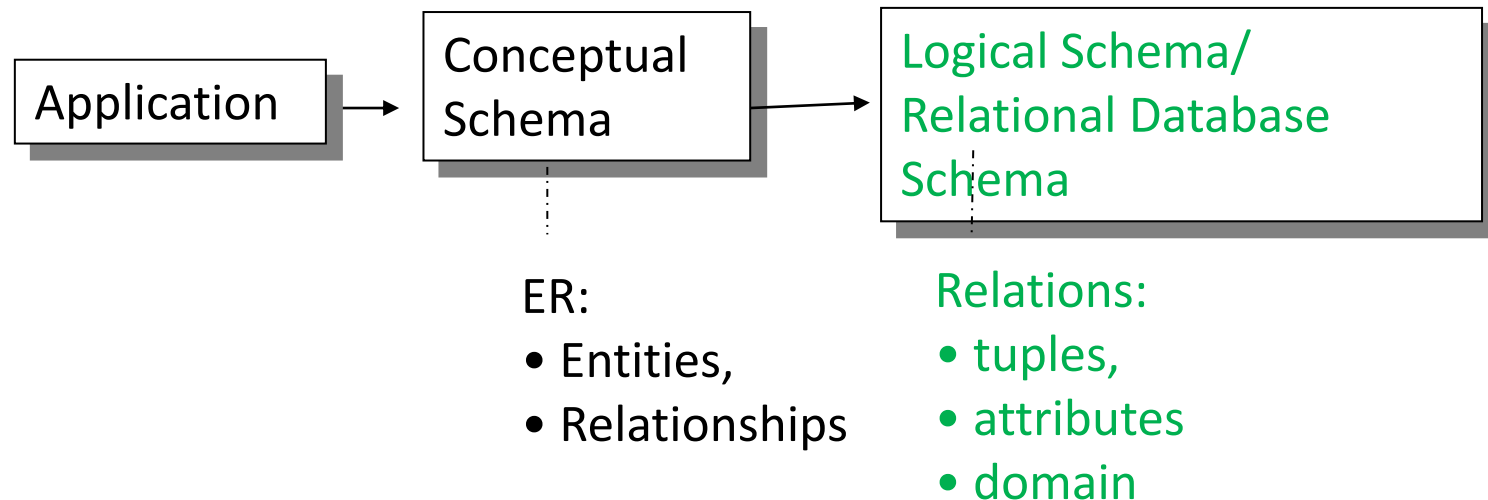


- Currently still the most popular data model for database management systems
 - Most popular from 1970 till now?! (How?)

- English computer scientist [Edgar F. Codd](#)

A Relational Model of Data for Large Shared Data Banks (1970)

<https://www.seas.upenn.edu/~zives/03f/cis550/codd.pdf>



Relational Data Model

- Formally, **relation schema** R is a finite set of attribute names, $R = (A_1, A_2, \dots, A_n)$
 - For each **attribute** name, A_i , has corresponding **domain** of A_i , denoted D_i , which is the **set of values** A_i can take
 - Given **domains** $D_1, D_2, \dots, D_n$, A **relation** r on relation schema R is a subset of $\mathbf{D} = D_1 \times D_2 \times \dots \times D_n$
- A relation r is a set of n -tuples $(a_1, a_2, \dots, a_n)$, where each $a_i \in D_i$
 - Each tuple t is an element of r , “a row in the table”
- E.g., *Students*(sid: string, name: string, login: string, age: integer, gpa: real)

Students	<u>sid</u>	name	login	age	gpa
----------	------------	------	-------	-----	-----

Relations - Unordered Set of Tuples

- Relation model requires that no two rows be identical
 - (In practice, some commercial systems may allow tables to have duplicate rows)
 - Defined to be a set of unique tuples or rows
 - The order in which the rows are listed is not important
 - The order of the fields is not important too
- Why is the order of tuples irrelevant?
 - An unordered collection of elements is a set
$$\{1, 2, 3\} = \{2, 1, 3\}$$
 - An ordered collection of elements is a list
$$(1, 2, 3) \neq (2, 1, 3)$$
 - A set expresses membership.
 - *E.g., we care you are a student, but we don't care whether you're the 6th student to register (the order)*

Relations - Unordered Set of Tuples

- The below are mathematically **same** relation.
 - The ordering of column (or rows) are irrelevant.

PLAYER					
Name	Position	Goals	Age	Height	Weight
Heady	Half-forward	17	24	183	83
Sumich	Full-forward	59	26	191	92
Langdon	Utility	23	23	189	86

PLAYER					
Name	Age	Height	Weight	Goals	Position
Sumich	26	191	92	59	Full-forward
Langdon	23	189	86	23	Utility
Heady	24	183	83	17	Half-forward

Concept of Keys Persists

- *Below, let's **assume** we are the first humans, and no two people have ever had the same name yet*
 - *Player (Name: string, Position: string, Goals: integer)*

PLAYER					
<u>Name</u>	Position	Goals	Age	Height	Weight
Heady	Half-forward	17	24	183	83
Sumich	Full-forward	59	26	191	92
Langdon	Utility	23	23	189	86

- *In this case, a possibility is...*
 - {Name} is unique and therefore is a **candidate key** for PLAYER
 - {Name, Goals} is a super-key but not a **candidate key** (because {Name} is a key)

Perquisite: NULL

- In relational model, **NULL** values represent attributes whose values are unknown or do not exist for some entity instance. In general, it is a special value to **indicate a lack of value**.
- Null: Example Usage: COMP9311 Student Schema:
 - (zID, trimester id, ass1-mark, ass2-mark, exam-mark)
 - COMP9311 Student Instance/Fact
 - 21T2 = (z0000001, 21T2, 16/25, 18/25, 50/50)
 - Suppose student from 22T1? use NULL and update later?

The diagram illustrates the mapping from a relation name to a table and from attributes to table columns. An arrow labeled "Relation Name" points to the word "STUDENT". Another arrow labeled "Attributes" points to the column headers of the table. A bracket labeled "Tuples" points to the rows of the table.

Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Benjamin Bayer	305-61-2435	(817)373-1616	2918 Bluebonnet Lane	NULL	19	3.21
Chung-cha Kim	381-62-1245	(817)375-4409	125 Kirby Road	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	(817)749-1253	25	3.53
Rohan Panchal	489-22-1100	(817)376-9821	265 Lark Lane	(817)749-6492	28	3.93
Barbara Benson	533-69-1238	(817)839-8461	7384 Fontana Lane	NULL	19	3.25

Sample Relational Model

- What business scenario can you identify?
- How do we store relationships?

STUDENT:

<u>Person#</u>	Name
1	Dr C.C.Chen
3	Ms K.Juliff
4	Ms J.Gledill
5	Ms B.K.Lee

RESEARCHER:

<u>Person#</u>	Name
1	Dr C.C.Chen
2	Dr R.G.Wilkinson

COURSE:

<u>Department</u>	<u>Degree</u>
Psychology	Ph.D.
Comp.Sci.	Ph.D.
Comp.Sci.	M.Sc.
Psychology	M.Sc.

Enrolment Relationship - Relation

- Relationship instances are stored via their key

STUDENT:

<u>Person#</u>	Name
1	Dr C.C.Chen
3	Ms K.Juliff
4	Ms J.Gledill
5	Ms B.K.Lee

RESEARCHER:

<u>Person#</u>	Name
1	Dr C.C.Chen
2	Dr R.G.Wilkinson

COURSE:

<u>Department</u>	<u>Degree</u>
Psychology	Ph.D.
Comp.Sci.	Ph.D.
Comp.Sci.	M.Sc.
Psychology	M.Sc.

ENROLMENT:

<u>Enrolment#</u>	Supervisee	Supervisor	Department	Degree
1	1	2	Psychology	Ph.D.
2	3	1	Comp.Sci.	Ph.D.
3	4	1	Comp.Sci.	M.Sc.
4	5	1	Comp.Sci.	M.Sc.

Enrolment Relationship+ - Relation

- A more “good” relational design (thx to E/R design)* (relationships key comprised by inherent attributes: P.Key(s) of participating entities)

STUDENT:

<u>Person#</u>	Name
1	Dr C.C.Chen
3	Ms K.Juliff
4	Ms J.Gledill
5	Ms B.K.Lee

RESEARCHER:

<u>Person#</u>	Name
1	Dr C.C.Chen
2	Dr R.G.Wilkinson

COURSE:

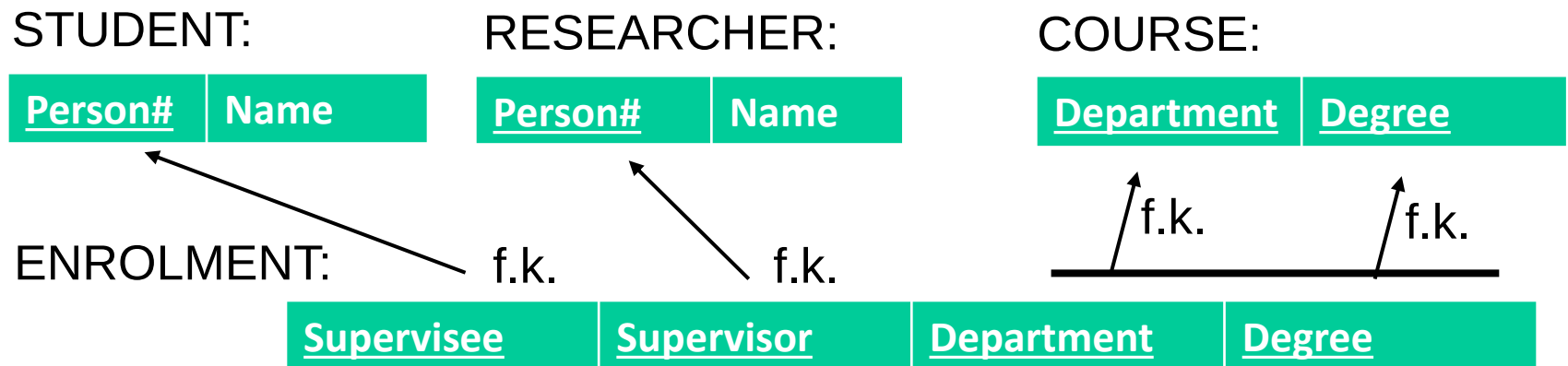
<u>Department</u>	<u>Degree</u>
Psychology	Ph.D.
Comp.Sci.	Ph.D.
Comp.Sci.	M.Sc.
Psychology	M.Sc.

ENROLMENT:

<u>Supervisee</u>	<u>Supervisor</u>	<u>Department</u>	<u>Degree</u>
1	2	Psychology	Ph.D.
3	1	Comp.Sci.	Ph.D.
4	1	Comp.Sci.	M.Sc.
5	1	Comp.Sci.	M.Sc.

Intro to Foreign Key

- **Foreign key**
 - *an attribute that keeps the value of a primary key of another relation.*



Relation Related to Other Relation(s)

- A set of attributes from a relation schema R_1 may be a foreign key, *FK*, if the following two conditions are true.
 - the attributes (from R_1) have **same domains** as attributes in the primary key of another relation schema R_2 , and
 - the value of FK in a tuple t_1 of R_1 either occurs as a value of PK for some tuple t_2 in R_2 or is null.
- It is a **constraint** on the schema.
 - E.g., For a relation with relation with a foreign key, when inserting new values into this, **each new tuple/record** should have...
 1. **primary-key** values not already present
 2. **foreign-key** values either **all null**, OR **all non-NULL AND occurs in the referenced relation**
 - If true, conditions remains fulfilled and insertion is permitted.

New Records! First Insertion

STUDENT:

<u>Person#</u>	<u>Name</u>
1	Dr C.C.Chen
3	Ms K.Juliff
4	Ms J.Gledill
5	Ms B.K.Lee

RESEARCHER:

<u>Person#</u>	<u>Name</u>
1	Dr C.C.Chen
2	Dr R.G.Wilkinson

COURSE:

<u>Department</u>	<u>Degree</u>
Psychology	Ph.D.
Comp.Sci.	Ph.D.
Comp.Sci.	M.Sc.
Psychology	M.Sc.

ENROLMENT:

<u>Supervisee</u>	<u>Supervisor</u>	<u>Department</u>	<u>Degree</u>
1	2	Psychology	Ph.D.
3	1	Comp.Sci.	Ph.D.
4	1	Comp.Sci.	M.Sc.
5	1	Comp.Sci.	M.Sc.

1. Insert < 2, Dr.V.Ciesielski > into RESEARCHER Allowed?

New Records! Second Insertion

STUDENT:

<u>Person#</u>	<u>Name</u>
1	Dr C.C.Chen
3	Ms K.Juliff
4	Ms J.Gledill
5	Ms B.K.Lee

RESEARCHER:

<u>Person#</u>	<u>Name</u>
1	Dr C.C.Chen
2	Dr R.G.Wilkinson

COURSE:

<u>Department</u>	<u>Degree</u>
Psychology	Ph.D.
Comp.Sci.	Ph.D.
Comp.Sci.	M.Sc.
Psychology	M.Sc.

ENROLMENT:

<u>Supervisee</u>	<u>Supervisor</u>	<u>Department</u>	<u>Degree</u>
1	2	Psychology	Ph.D.
3	1	Comp.Sci.	Ph.D.
4	1	Comp.Sci.	M.Sc.
5	1	Comp.Sci.	M.Sc.

2. Insert < Comp.Sci.,NULL > into COURSE Allowed?

New Records! Third Insertion

STUDENT:

<u>Person#</u>	<u>Name</u>
1	Dr C.C.Chen
3	Ms K.Juliff
4	Ms J.Gledill
5	Ms B.K.Lee

RESEARCHER:

<u>Person#</u>	<u>Name</u>
1	Dr C.C.Chen
2	Dr R.G.Wilkinson

COURSE:

<u>Department</u>	<u>Degree</u>
Psychology	Ph.D.
Comp.Sci.	Ph.D.
Comp.Sci.	M.Sc.
Psychology	M.Sc.

ENROLMENT:

<u>Supervisee</u>	<u>Supervisor</u>	<u>Department</u>	<u>Degree</u>
1	2	Psychology	Ph.D.
3	1	Comp.Sci.	Ph.D.
4	1	Comp.Sci.	M.Sc.
5	1	Comp.Sci.	M.Sc.

3. Insert <6, 2, Psychology, Ph.D. > into ENROLMENT Allowed?

Relational Integrity Constraints

- Suppose our data is originally in a ***valid state***:
- How might a valid relation become invalid?
 - Insertions on database can result in an invalid state!
- For each new tuple/record
 1. **primary-key** values not already present
 2. value of each **foreign-key** either **all null**, OR **all non-NULL AND occurs in the referenced relation**

Top 3 Integrity Constraints

- Three integrity constraints are important
 1. **Key constraint**: candidate key values must be unique for every relation instance.
 2. **Entity integrity**: an attribute that is part of a primary key cannot be NULL.
 3. **Referential integrity**: closely related to foreign keys
- “Valid” or “Invalid” state?
 - “Valid” state: relation does not violate any integrity constraint
 - “Invalid” state: relation violates at least one integrity constraint

Self-Reading: Deletion

- Record to delete could have foreign key values referring another entity from another relation.
 - can this cause an invalid state?
 - Example:
 - Delete tuple with Person# = 2 from RESEARCHER
 - Supervisor in ENROLMENT is FK -> person# in RESEARCHER

RESEARCHER:

<u>Person#</u>	Name
1	Dr C.C.Chen
2	Dr R.G.Wilkinson

ENROLMENT:

<u>Supervisee</u>	<u>Supervisor</u>	<u>Department</u>	<u>Degree</u>
1	2	Psychology	Ph.D.
3	1	Comp.Sci.	Ph.D.
4	1	Comp.Sci.	M.Sc.
5	1	Comp.Sci.	M.Sc.

- Mental Exercise: What are the complications?

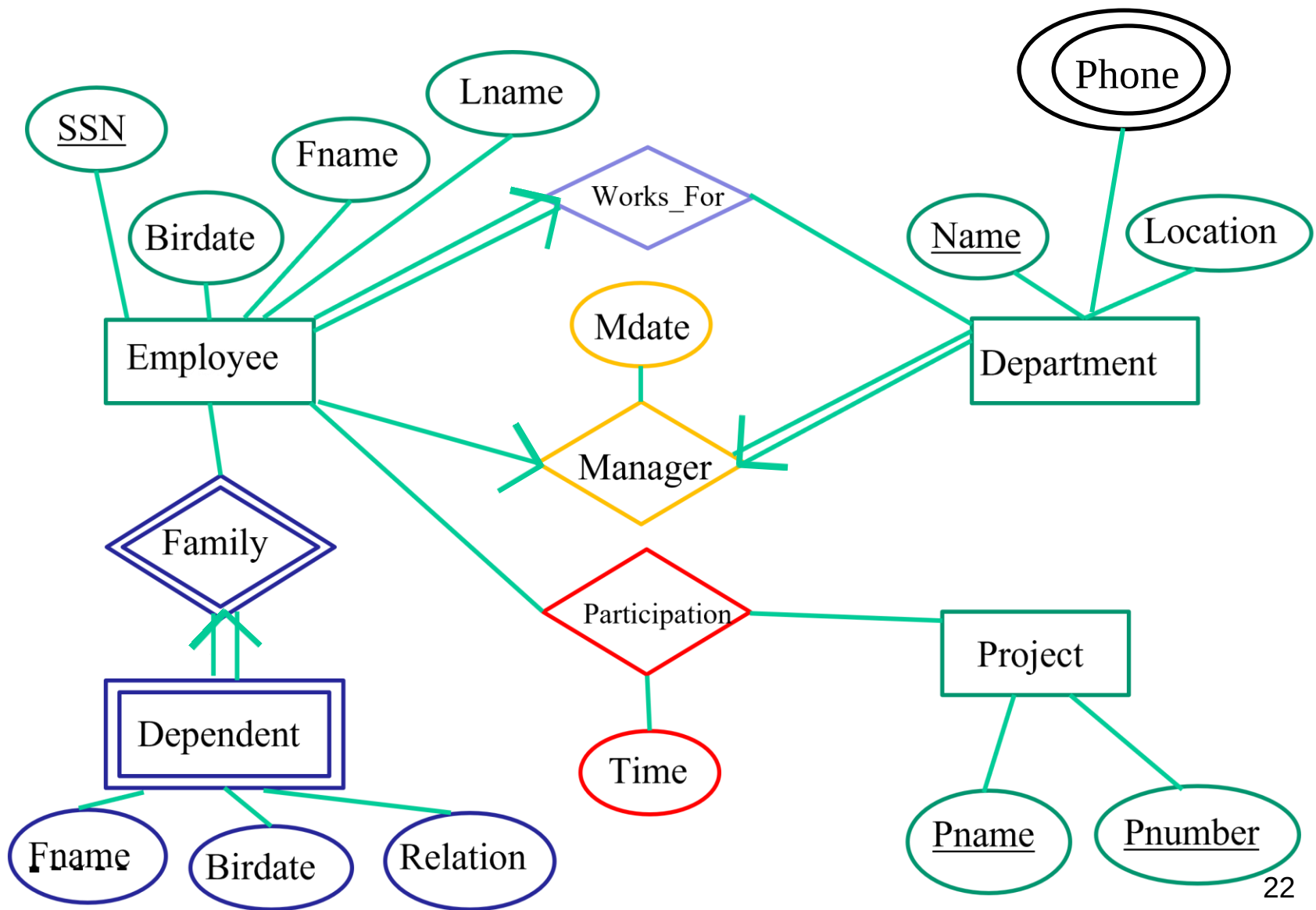
Self-Reading: Updates

- Can updating existing foreign key values lead to an invalid state?
 - not unless you're modifying the **value of a key**
- If the modified attribute is the relation's primary key
 - same issues as deleting PK1 then immediately inserting PK2
 - make sure deletion and insertion don't violate any steps
- If the modified attribute is part of a foreign key
 - check that the new value refers to an existing tuple

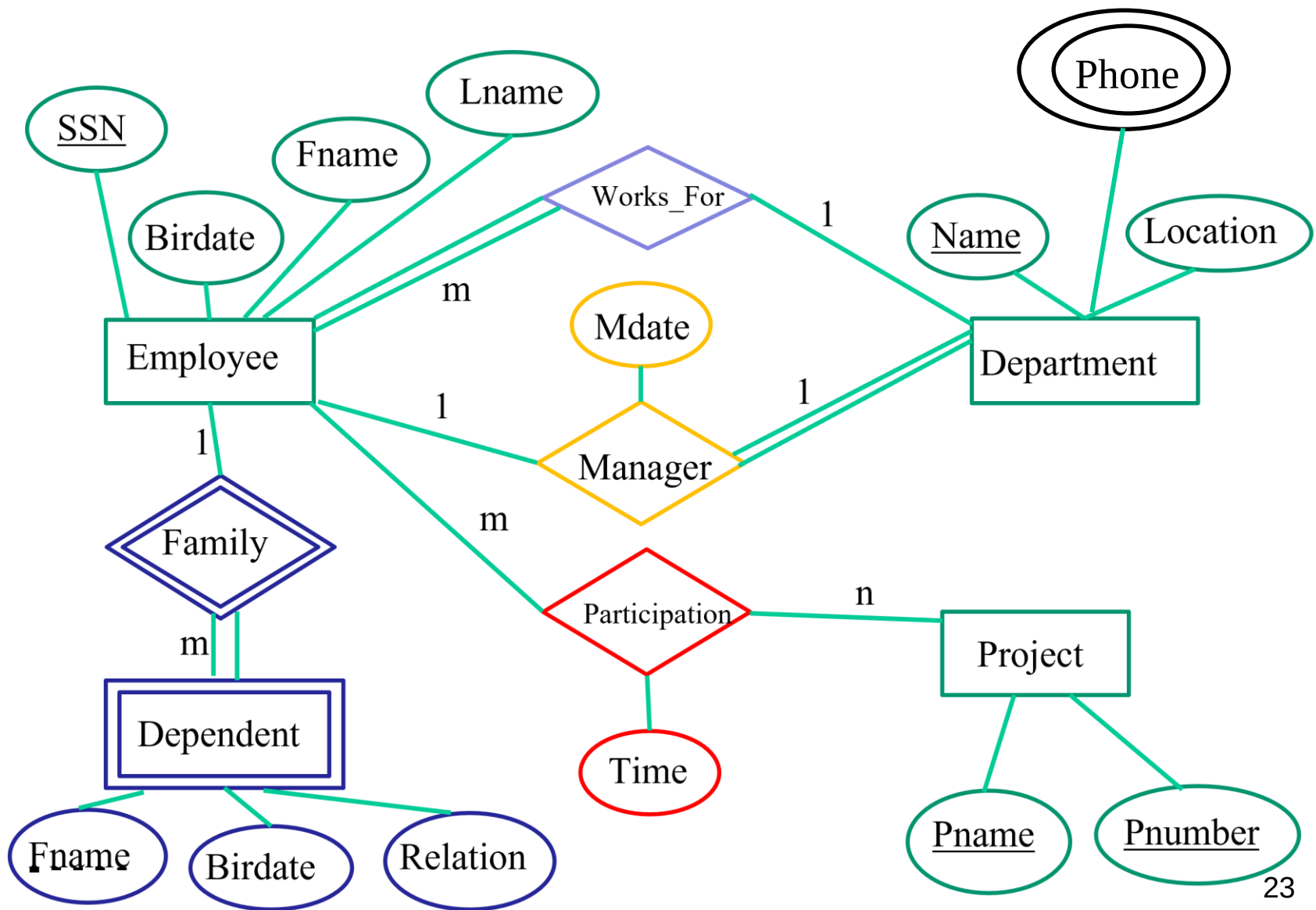
(2) Given E/R model, effective
translation into "good" relational model

Deriving a Relational Diagram from an ER Diagram

Motivating Business Scenario



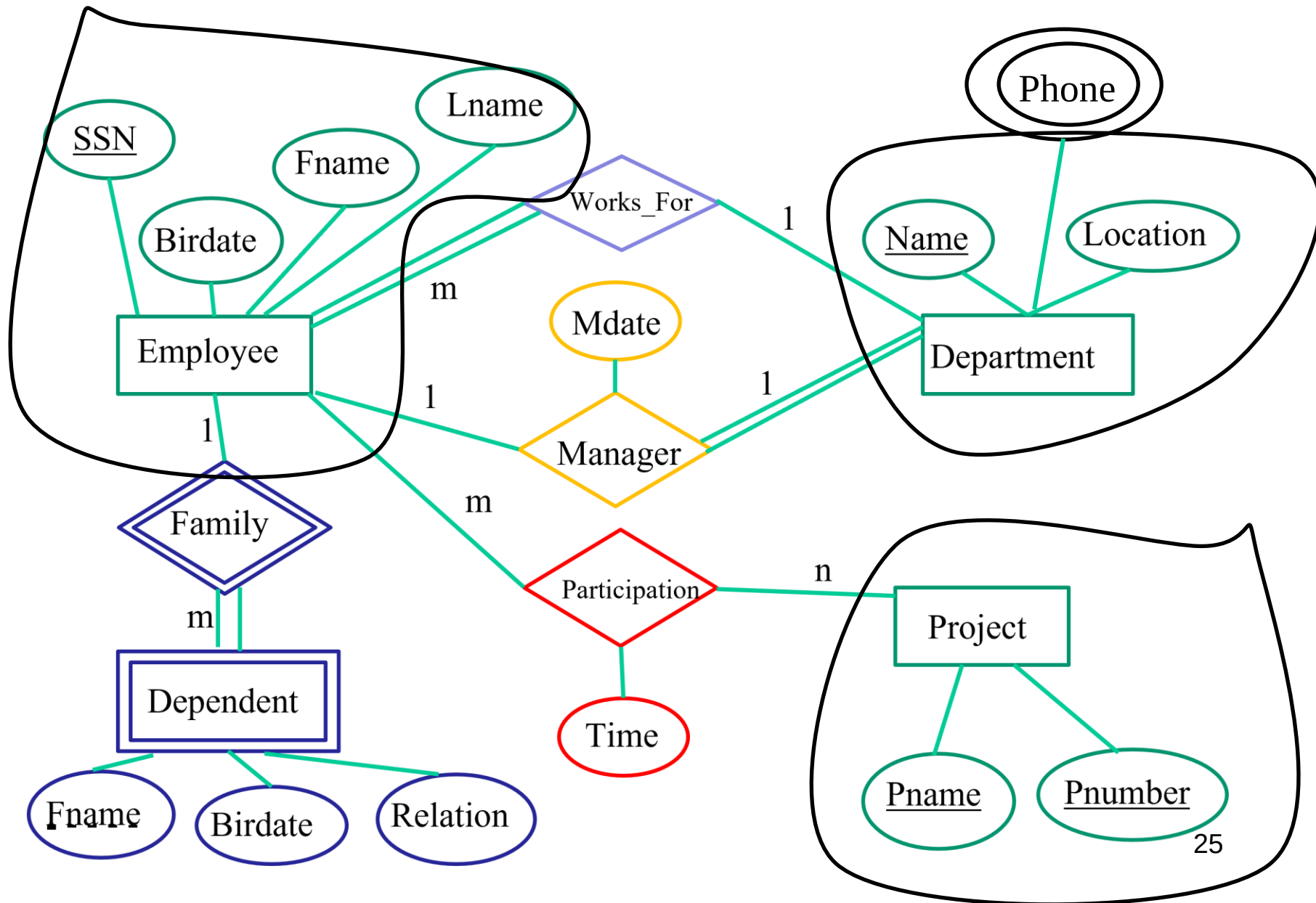
**Below, Chen's notation: only difference is in cardinality representation.
Used here to allow us to focus on the translation aspects first.**



Step 1: Strong Entities

- For each ***strong/regular entity*** (not weak entity) type E:
- Create a **new** relation R with
 - **Attributes of R:** all simple attributes (and simple components of composite attributes) of E
 - **Key of R:** key of E as the primary key for the relation

Step 1: Strong Entities



Step 1: Strong Entities

Employee

<u>SSN</u>	Fname	Lname	Birdate
------------	-------	-------	---------

Department

<u>Name</u>	Location
-------------	----------

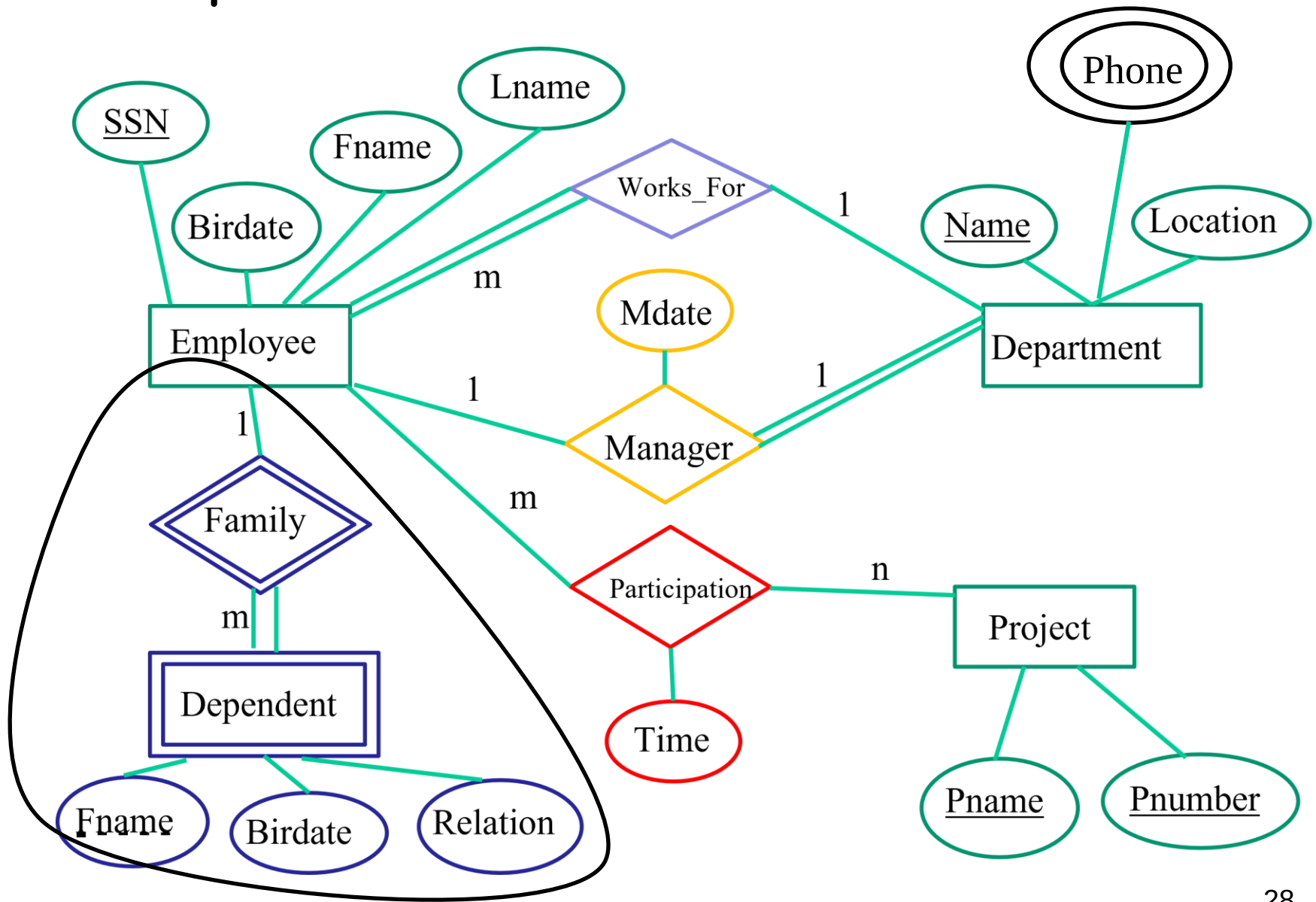
Project

<u>Pname</u>	Pnumber
--------------	---------

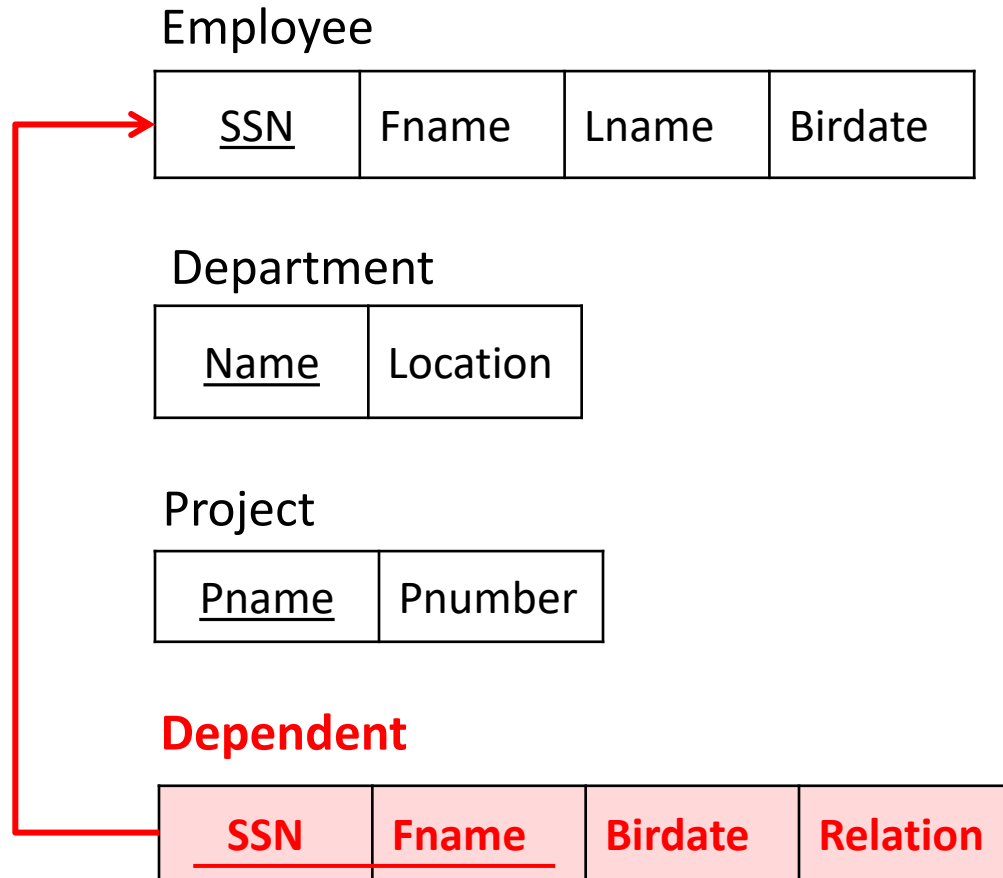
Step 2: Weak Entities

- For each *weak entity type* W with the owner entity type E:
- Create a **new** relation R with
 - **Attributes of R:**
 - all simple attributes (and simple components of composite attributes) of W
 - primary key attributes of relation derived from E
 - **Key of R:** foreign key to E and partial key of W

Step 2: Weak Entities



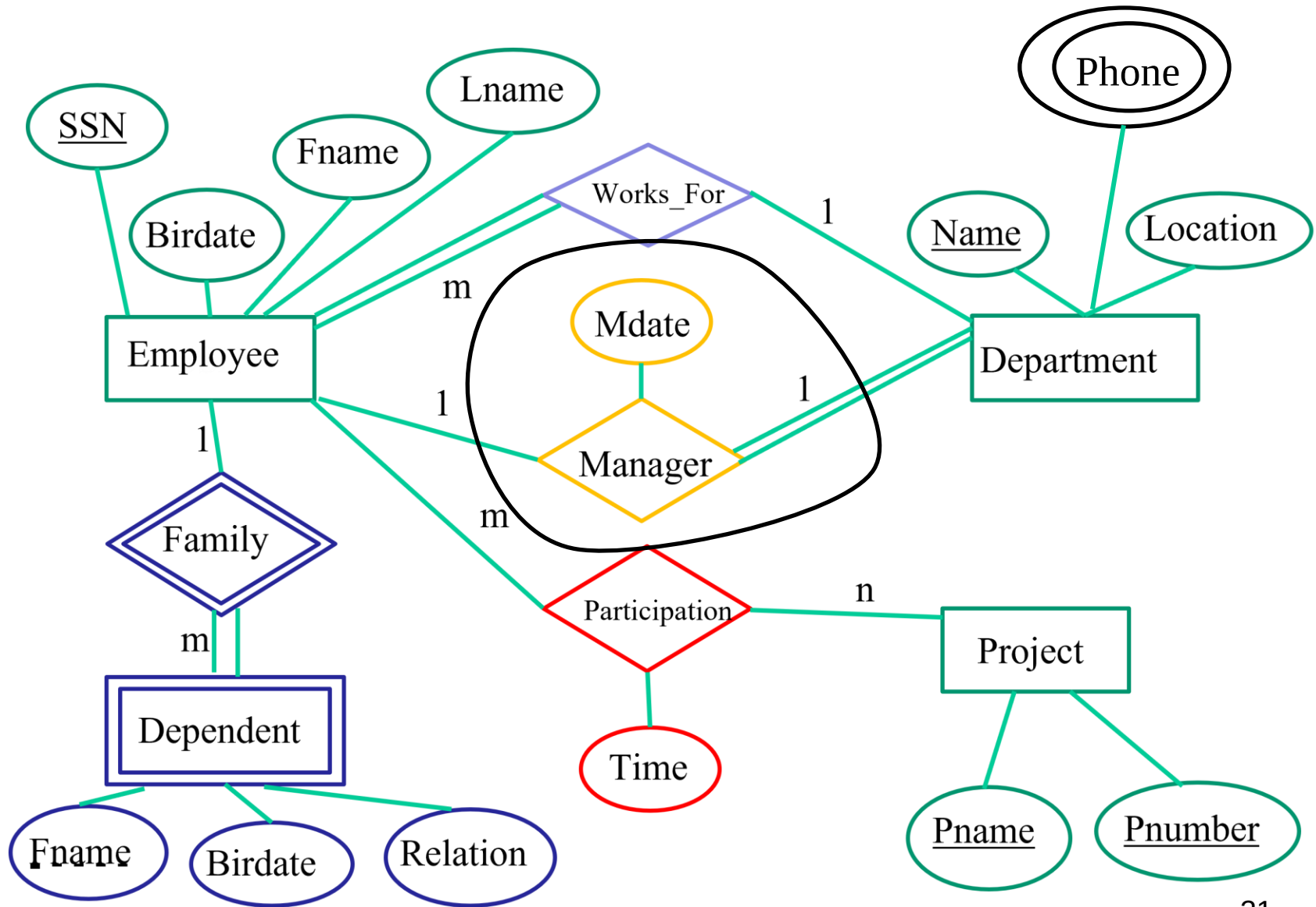
Step 2: Weak Entities



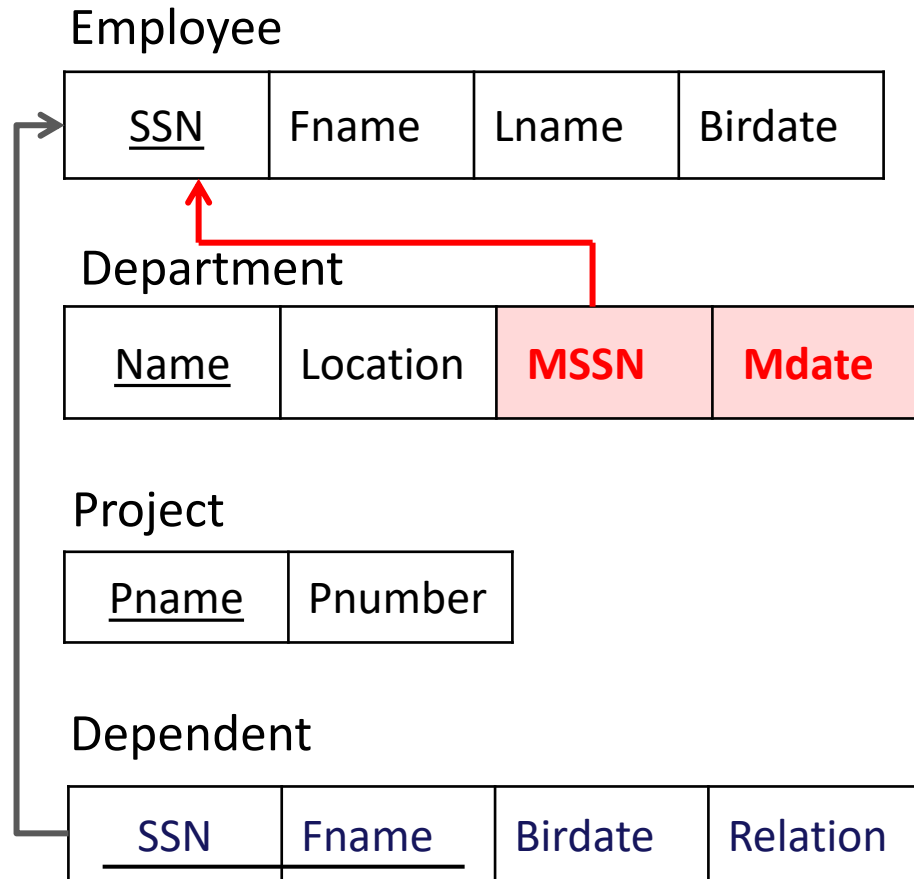
Step 3 - 1:1 Relationships

- For each **1:1 relationship type** B. Let S and T be the participating entity types
- Steps
 - Choose one of S and T (let S be one that participates totally if there is one)
 1. Add attributes from primary key of T to S as a foreign key
 2. Add all simple attributes (and simple components of composite attributes) of B as attributes of S

Step 3 - 1:1 Relationships



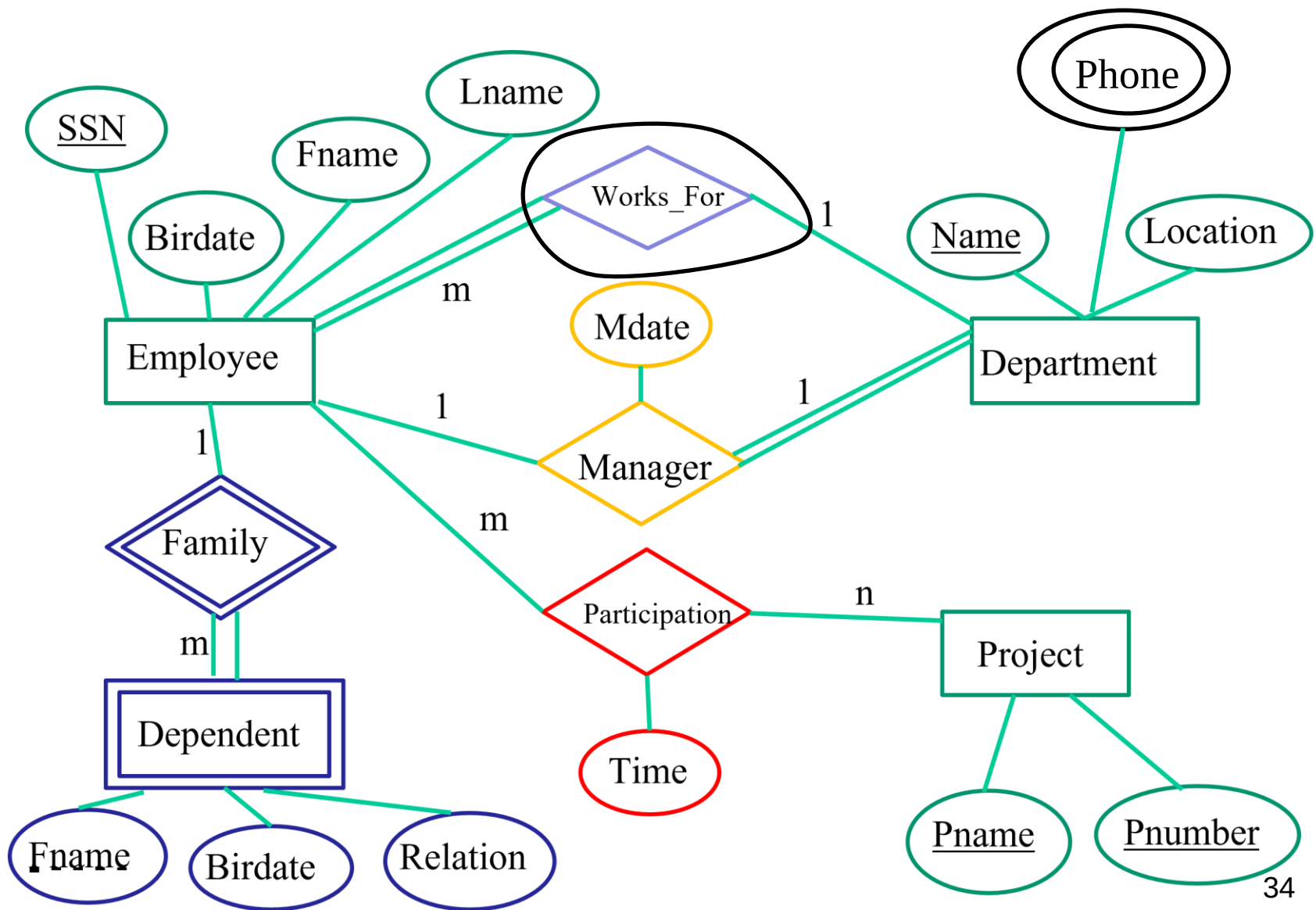
Step 3 - 1:1 Relationships



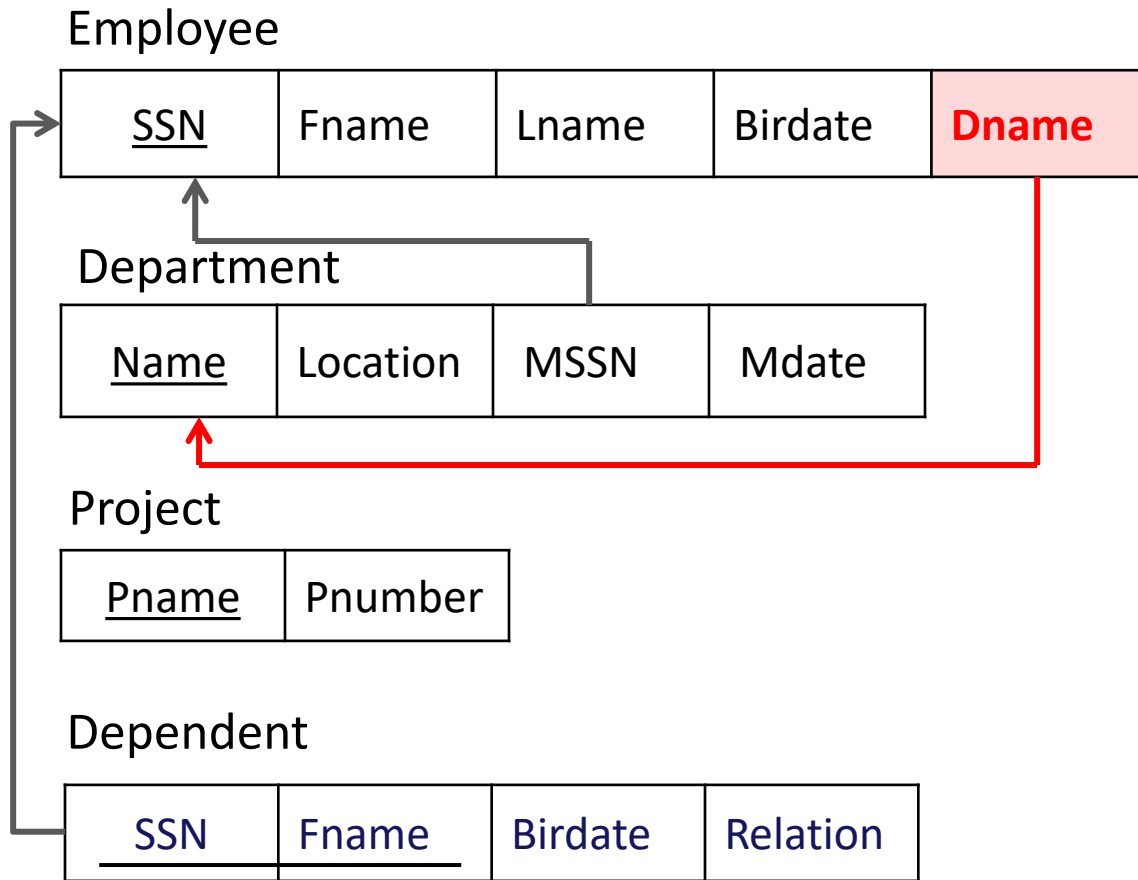
Step 4 – 1:M Relationships

- For each **1:N relationship type** B. Let *S* and *T* be the participating entity types, where *S* is on the 1 side and *T* on the N side
- Steps
 - Add to relation belonging to entity T
 - attributes from primary key of *S* as a foreign key
 - any simple attributes (or simple components of composite attributes) from relationship B
 - *(This step doesn't add any new tuples, just attributes.)*

Step 4 - 1:M Relationships



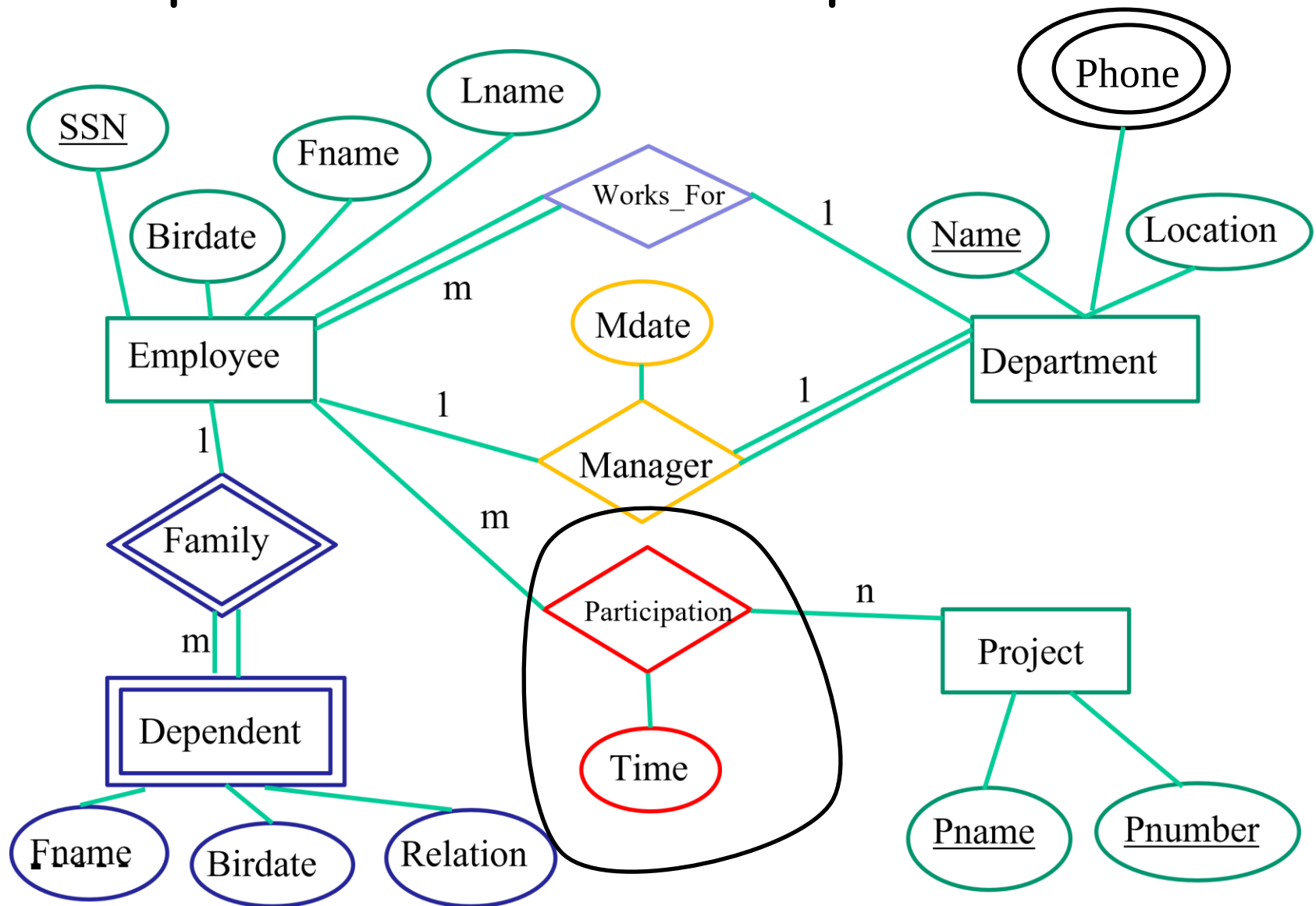
Step 4 - 1:M Relationships



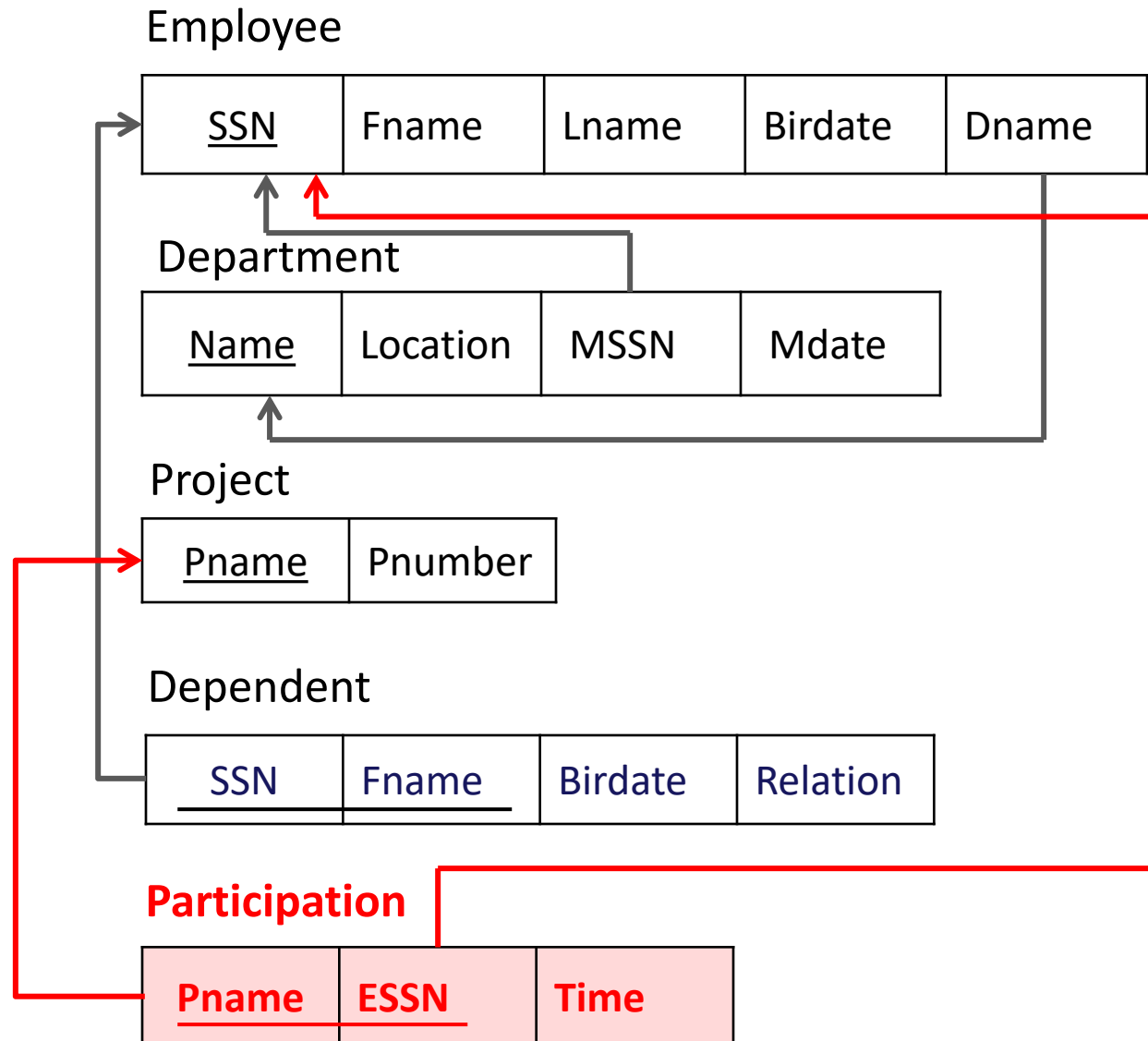
Step 5 - M:N Relationships

- For each ***N:M relationship type*** B. Let *S* and *T* be the participating entity types
- Create a **new** relation R with
 - **Attributes of R:**
 - Attributes from key of S as foreign key
 - Attributes from key of T as foreign key
 - Simple attributes, and simple components of composite attributes of relation B
 - **Key of R:** All attributes from key of S and key of T

Step 5 - M:N Relationships



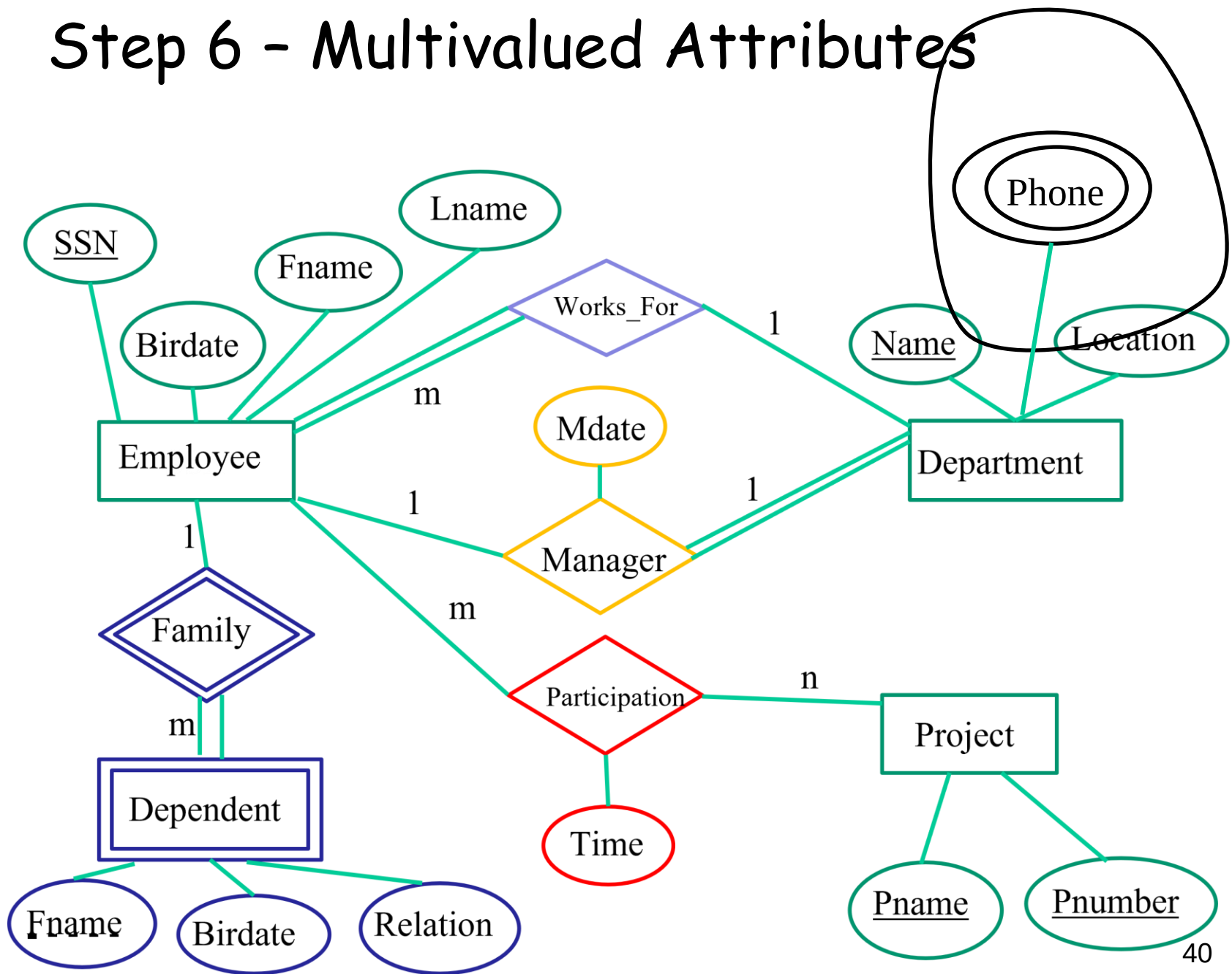
Step 5 - M:N Relationships



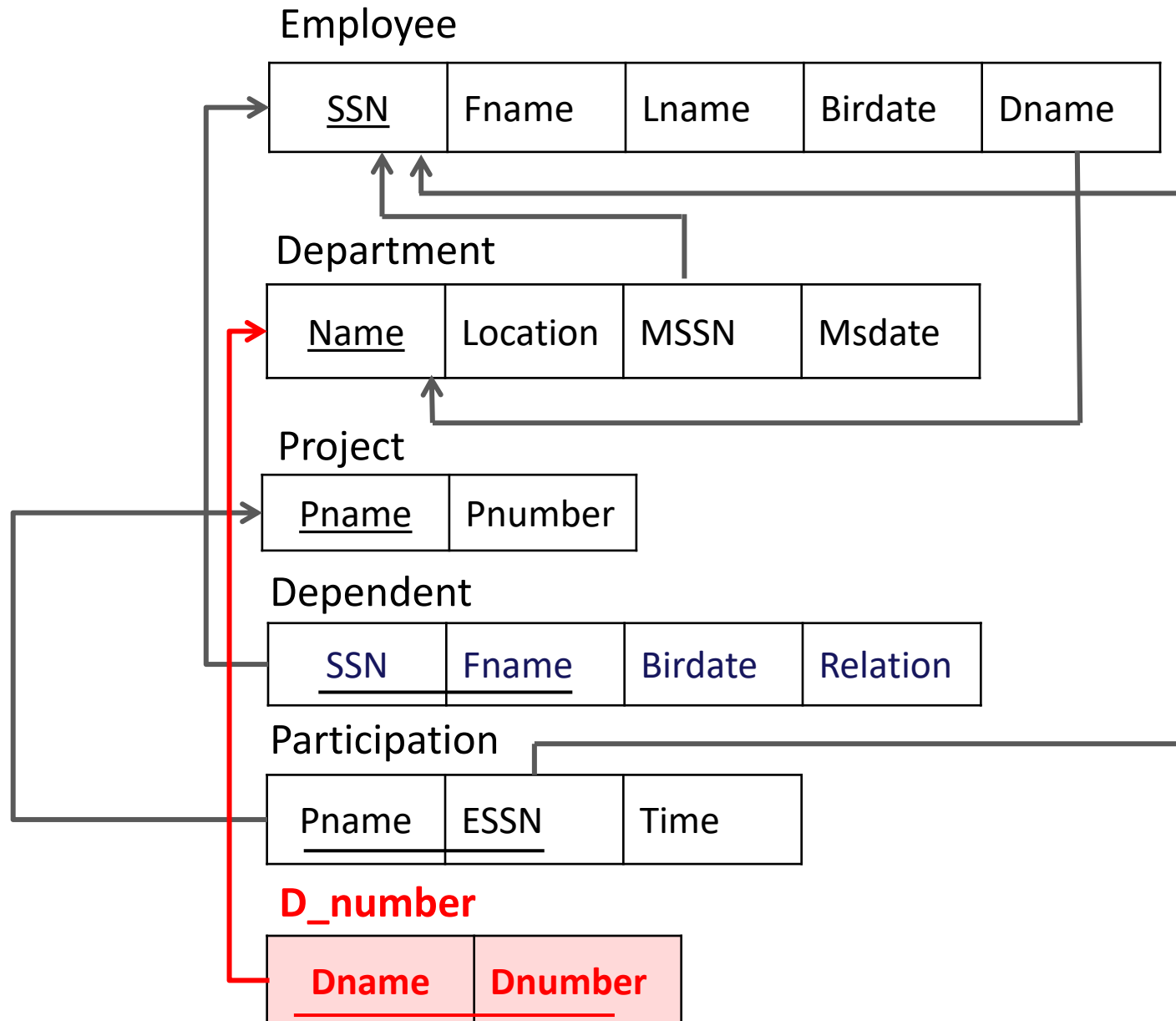
Step 6 - Multivalued Attributes

- For each ***multivalued attribute*** A, where A is an attribute of E, create a **new** relation R
- *If A is a multivalued simple attribute,*
 - Attributes of R = Simple attribute A, and key of E as a foreign key
- *If A is a multivalued composite attribute,*
 - Attributes of R = All simple components of A, and key of E as a foreign key
- In both cases, the primary key of R is the set of all attributes in R.

Step 6 - Multivalued Attributes



Step 6 - Multivalued Attributes

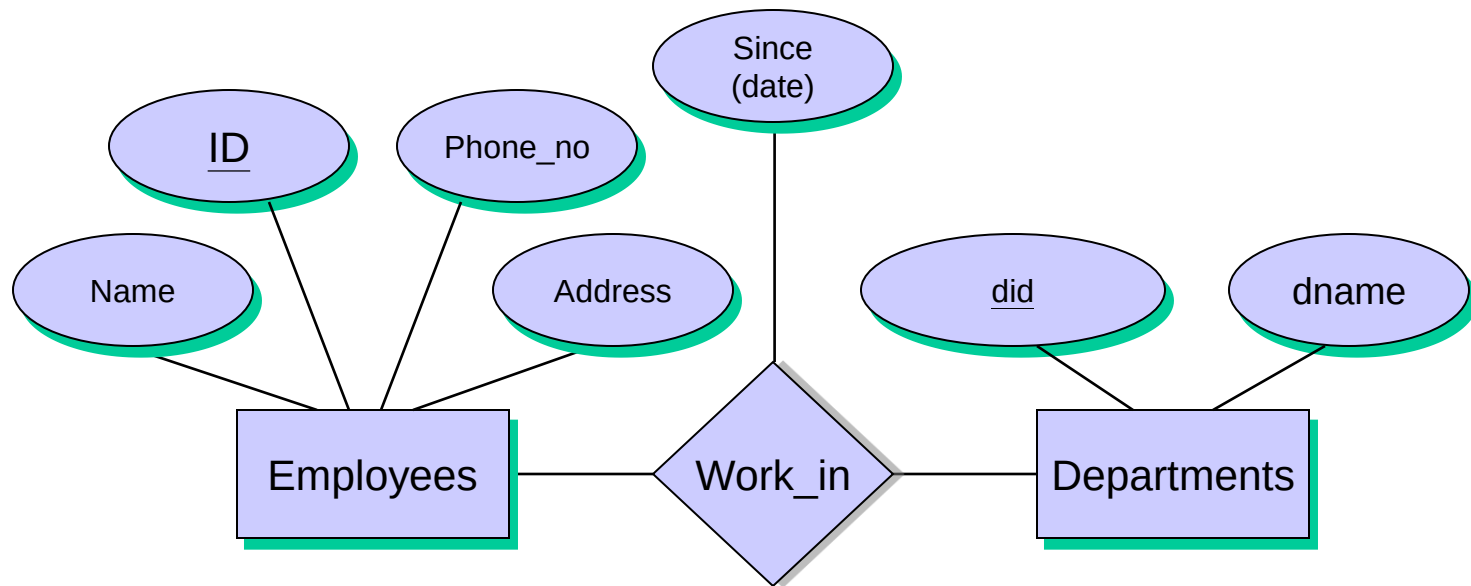


Self Reading: On N-ary Relationships

- For each *n-ary relationship type* ($n > 2$)
- Create a **new** relation R with
 - Attributes : **same for mapping M:N.**
 - Key :
 - Same for mapping M:N, see exception below
 - The exception is that that if one of the participating entity types has participation ratio 1, its key can be used as a key for the new relation.
- *(Advice: binary relationships simpler to model)*

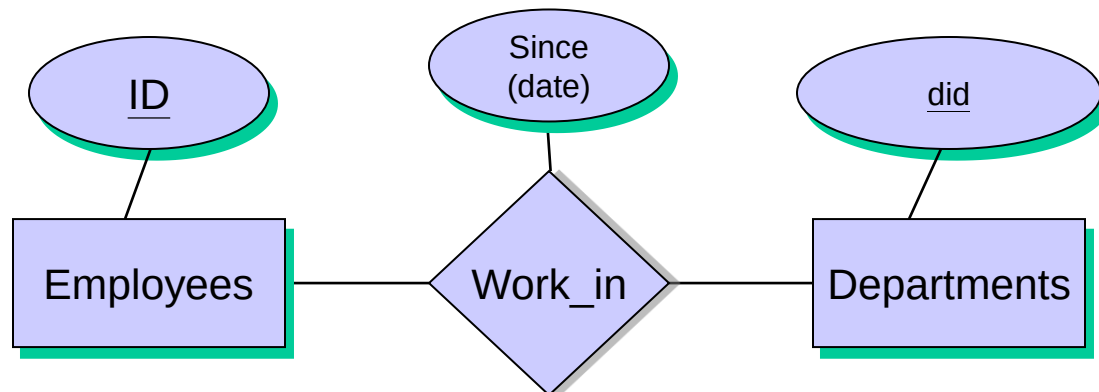
Keys in Relationships (0)

- There is also language relating to a relationship set.
 - A relationship must be uniquely identified. The participating entity instances contribute to identification of the relationship instance .



Keys in Relationships (1)

- Here, each relationship must be uniquely identified by the combination of the employee *hkid* and the department *did*.
 - Meaning for each employee-department pair, we cannot have more than one associated “since” value.



Keys in M-N Relationships (2)

- **Many-to-many**
 - For a relationship among $E_1, \dots, E_k$, with no mapping constraint, the primary key is normally the union of the primary keys for $E_1, \dots, E_k$
 - *E.g.*, $\{ID, loan_number\}$ is the key for borrower
 - (For time being, let's only consider value of $k = 2$)
- **One-to-many** (or many-to-one [same concept])
 - An entity set E has a key constraint in a relationship set R , such that each entity in an instance of E appears in at most one relationship in instance of R
 - Hence **an entity in E can uniquely identify a relationship in R**
 - Thus, key of E can be used as the key in R
 - *e.g.* child-mother: is a many-to-one relationship, where entity Child has a key constraint. **child can be the primary key in the relationship set**

Keys in Relationships (4)

- Concept of keys is also used to identify a relationship
- **One-to-one**
 - For a one-to-one relationship between two entity sets E and F , $\text{key}(E)$ and $\text{key}(F)$ are both keys for relationship set
 - e.g. “manages” relation both $\{ID\}$ and $\{did\}$ are keys of relation
 - Recall mapping:
 - Choose one of S and T (let S be one that participates totally if there is one)
 1. Add attributes from pkey of T to S as a foreign key
 2. Add all simple attributes (and simple components of composite attributes) of B as attributes of S

References

- Some slides are inspired by my own teaching materials in the past.